MCP Agora

MCP Server with cross-agent persistent memory for AI agent fleets

Antonio Cioffi

2025

MCP Agora is a portfolio/learning project implementing an MCP Server (Model Context Protocol) with cross-agent persistent memory for personal AI agent fleets. It uses ChromaDB for vector indexing, sentence-transformers for semantic embeddings (all-MiniLM-L6-v2 model, 384 dimensions), SQLite for metadata and persistent cache, and FastMCP to expose 8 MCP tools. The architecture is organized into 7 layers (Transport, Protocol, Router, Memory, Cache, Backend Connector, Embedding) with semantic routing to external MCP backends (GitHub, Playwright). The project has 61 automated tests across 10 files, zero mocks, using real ChromaDB and SQLite in isolated temp directories.

Repository: https://github.com/cioffiAI/mcp-agora

1. Introduction and Goals

MCP Agora is a portfolio/learning project implementing an MCP Server (Model Context Protocol) with cross-agent persistent memory for personal AI agent fleets. It stems from a real need to solve a concrete problem: modern AI agents (Claude Code, Codex CLI, ChatGPT, Gemini CLI) operate in isolated sessions with no shared memory. Each agent repeats searches, re-contextualizes information, and duplicates work that another agent has already done.

1.1 Product Goals

- One agent saves knowledge -> another retrieves it (zero duplication)
- A query already made -> cached response (zero recomputation)
- A mistake already made -> not repeated (decision memory)
- 4+ agents sharing context without individual configuration

1.2 Learning Goals

- MCP Protocol deep dive: compliant MCP Server implementation using official SDK
- Vector search & embeddings: ChromaDB + sentence-transformers integration
- Layered system design: modular architecture with 7 separate layers and abstract interfaces
- Async Python: asyncio for MCP transport concurrency
- Advanced SQLite: relational schema for metadata, provenance, analytics

- Caching strategies: in-memory TTLCache + persistent SQLite

1.3 Positioning

Agora does not compete with enterprise projects like ContextForge (IBM), MetaMCP, AutoMem, or mcp-memory-service. It is a portfolio project demonstrating competence in AI infrastructure, MCP protocol, vector search, semantic routing, and system design, solving a real problem in personal agent workflows. Its value lies not in concept originality, but in execution quality: 2026 stack (MCP, vectors, embedding), clean architecture (7 layers, abstract dependencies, testable), and a real problem (isolated agent memory).

2. Technology Stack

The project is entirely developed in Python 3.13+ with uv as package manager. The following table summarizes the main stack components.

ing	Client)	**e**

2.1 Technology Choices

FastMCP
The official Anthropic SDK offers FastMCP, a high-level wrapper that simplifies MCP server creation. Compared to low-level Server, it requires less boilerplate and allows focusing on application logic.

ChromaDB
Embedded vector database, zero config, zero cloud. Supports ANN index (HNSW) for O(log n) search and cosine distance. PersistentClient ensures disk persistence without external server.

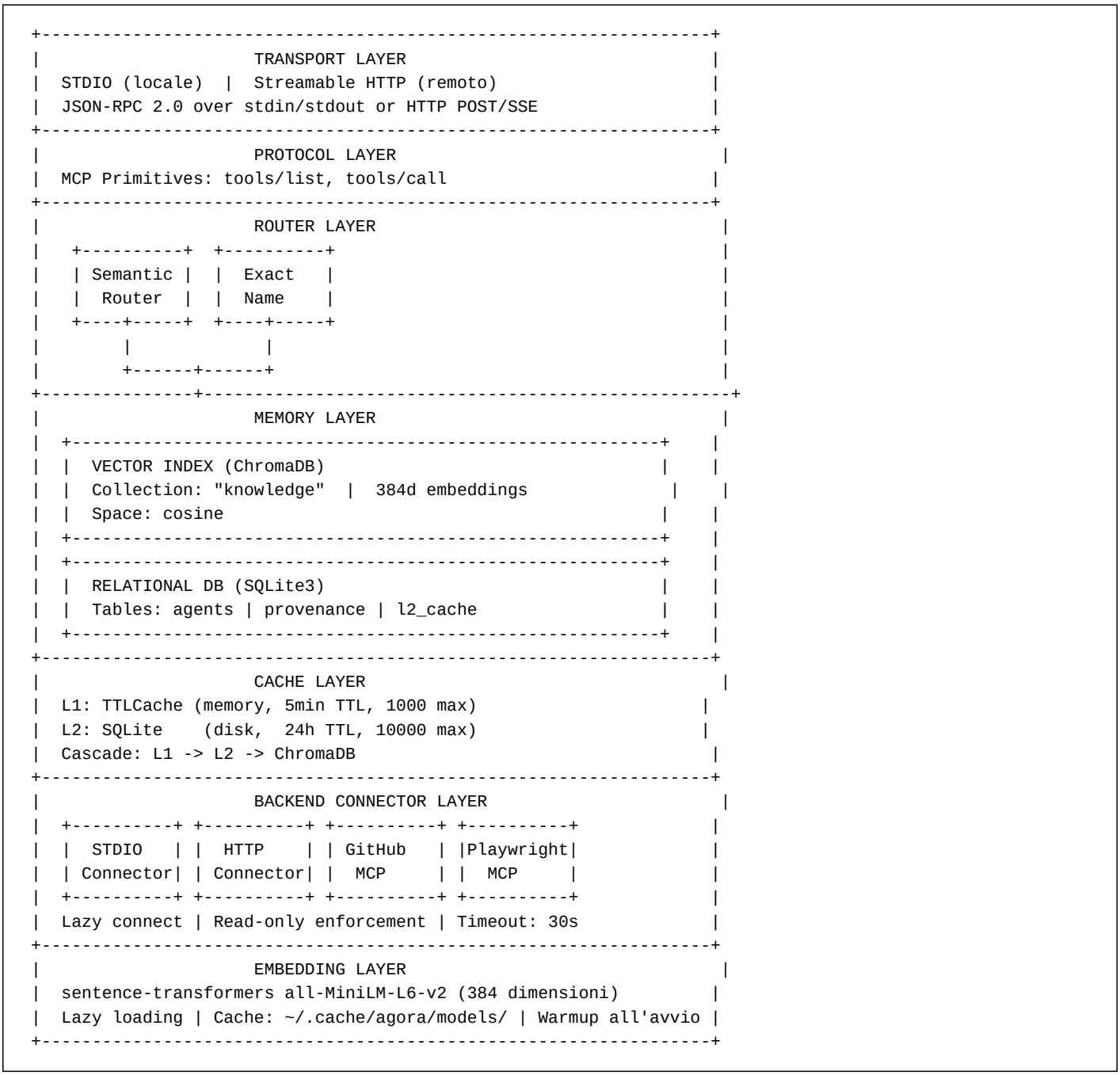
sentence-transformers
all-MiniLM-L6-v2 model: 384 dimensions, ~80MB, sufficient for MVP. Lazy-loaded (not at import) and cached in ~/.cache/agora/models/.

SQLite
For metadata, provenance and L2 cache. Zero configuration, file-based, included in Python standard library. Each operation opens and closes a connection (thread-safe).

3. Architecture

MCP Agora's architecture is organized into 7 separate layers, each with well-defined responsibilities and abstract interfaces. This modular approach allows testing, replacing, and evolving each layer independently.

3.1 Architecture Diagram



3.2 Layer Description

Layer 1: Transport Layer

Manages communication with agents via JSON-RPC 2.0. Supports two transports: STDIO (local, stdin/stdout, default) for agents like Claude Code and Codex CLI; Streamable HTTP (remote, HTTP POST + SSE) for remote or multi-session agents.

Layer 2: Protocol Layer

Exposes standard MCP primitives: tools/list to discover available tools, tools/call to invoke them. Agora exposes 8 tools (see Section 4).

Layer 3: Router Layer

Decision-making core of the system. When an agent calls agora_route, the router first attempts an exact name match (case-insensitive), then a semantic match via cosine similarity on backend description embeddings (threshold >= 0.5). If no match, returns error. Broadcast routing (agora_broadcast) sends the request in parallel to all backends, limited to read-only tools only.

Layer 4: Memory Layer

Composed of two sub-layers: Vector Index (ChromaDB, 'knowledge' collection, cosine space, 384 dimensions) for semantic search, and Relational DB (SQLite3, tables agents, provenance, l2_cache) for structured metadata. Knowledge retention is permanent (until deleted with agora_forget). L2 cache has configurable TTL (default 24h).

Layer 5: Cache Layer

Two cascading cache levels. L1: cachetools TTLCache in-memory (1000 entries, 5 minutes TTL). L2: SQLite-backed on disk (10000 entries, 24h TTL). Cache key = SHA256 of full JSON-serialized request with sort_keys=True. On agora_save and agora_forget, both caches are invalidated (full clear). Cascade: L1 -> L2 -> ChromaDB. On ChromaDB miss, both caches are populated.

Layer 6: Backend Connector Layer

Each external MCP backend is represented by a connector implementing the abstract BackendConnector interface. Two implementations: StdioConnector (subprocess MCP via stdio_client + ClientSession, lazy connect, asyncio.timeout) and HttpConnector (streamable HTTP MCP via streamablehttp_client). Connection is lazy: the connector connects on first use, not at startup. Read-only enforcement: if a backend is marked read_only, only tools with whitelisted prefixes (list_, get_, fetch_, read_, search_, find_, query_, describe_, show_) can be called. Mutative tools raise ReadOnlyBlockedError.

Layer 7: Embedding Layer

Abstract EmbeddingProvider with concrete SentenceTransformerProvider implementation using all-MiniLM-L6-v2 (384 dimensions). Lazy loading: model loads on first use, not on module import. Cache in ~/.cache/agora/models/. First call ~17s (model download + torch). Subsequent calls <1s. The warmup() method pre-loads the model at server startup.

4. API Tools

Agora exposes 8 MCP tools, registered via @mcp.tool() decorator on FastMCP. All tools accept and return JSON-serializable dictionaries.

Tool	Parametri	Comportamento	Cache
agora_save	content, tags, agent, session, confidence	Salva memoria in ChromaDB + provenance in SQLite. Registra agente.	Clear L1+L2
agora_query	query, top_k (=5)	Ricerca semantica. Cascade: L1 -> L2 -> ChromaDB.	Popola L1+L2 su miss
agora_route	target, tool, arguments	Routing a backend MCP. Exact match -> semantic (>=0.5).	No
agora_broadcast	tool, arguments	Broadcast parallelo (asyncio.gather) a tutti i backend. Solo read-only.	No
agora_backends	(none)	Lista backend configurati + stato connessione.	No
agora_crossref	query, entry_id, top_k	Correlazioni cross-agente. Per query: raggruppa per agente. Per ID: trova entry correlate.	No
agora_forget	entry_ids, tags, agent, dry_run	Elimina memoria. Filtri compositi. dry_run preview. Clear L1+L2.	Clear L1+L2
agora_status	(none)	Statistiche: entry, agenti, cache L1+L2, backend, db_size.	No

5. Data Schema

5.1 ChromaDB: Vector Store

Single 'knowledge' collection with cosine metric space. Documents are saved texts, embeddings are 384-dimensional vectors from all-MiniLM-L6-v2, and metadata includes tags (CSV format), created_at (ISO 8601), and agent.

The VectorStore wrapper (agora/memory/vector_store.py) abstracts the ChromaDB API and handles collection creation and serialization automatically.

5.2 SQLite: Relational Database

Three tables for structured metadata: agents (agent registry), provenance (knowledge entry traceability), l2_cache (persistent disk cache).

```
CREATE TABLE IF NOT EXISTS agents (
    id              TEXT PRIMARY KEY,
    name            TEXT NOT NULL,
    agent_type      TEXT DEFAULT 'unknown',
    first_seen      TEXT NOT NULL,
    last_seen       TEXT NOT NULL,
    sessions_count  INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS provenance (
    entry_id        TEXT NOT NULL,
    source_agent    TEXT NOT NULL,
    source_session  TEXT NOT NULL,
    confidence      REAL DEFAULT 0.5,
    created_at      TEXT NOT NULL,
    PRIMARY KEY (entry_id, source_agent, source_session)
);

CREATE TABLE IF NOT EXISTS l2_cache (
    cache_key       TEXT PRIMARY KEY,
    result_json     TEXT NOT NULL,
    ttl_seconds     INTEGER NOT NULL DEFAULT 86400,
    hit_count       INTEGER DEFAULT 0,
    created_at      TEXT NOT NULL,
    expires_at      TEXT NOT NULL
);
```

6. Testing

61 tests across 10 files, zero mocks - real ChromaDB and SQLite in isolated temp directories per test (pytest function-scoped fixtures with temp directory). Timing not used as metric (flaky on Windows); cache.stats()['hit_count'] is used instead.

[illegible]	[illegible]	[illegible]allback

7. Design Decisions

The following design decisions were made consciously, with explicit rationale. They should not be changed without discussion.

[illegible]	[illegible]enza

8. Conclusions and Roadmap

8.1 Completed Roadmap

Fase 1 - Core Memory
save/query/status with ChromaDB, sentence-transformers, TTLCache L1, FastMCP.

Fase 2 - Routing + Connectors
Semantic + exact Router, StdioConnector, HttpConnector, BackendRegistry, broadcast, read-only enforcement.

Fase 3 - Cross-Agent Memory
Provenance tracking (SQLite), persistent L2 cache, crossref, forget, agent/session/confidence params on save.

Fase 4 - Robustness
Structured logging, backend health checks (healthy/unhealthy/dead), configurable retry, rate limiting, non-blocking

startup, WarmingUpError grace.

8.2 Future Roadmap

- Phase 5: Full README, real examples, GitHub + PyPI publication (in progress)
- Chunking: multi-paragraph document handling with semantic boundary split

8.3 Success Metrics

[illegible]	[illegible] (target)

MCP Agora is not an invention. It is an engineering exercise on a real problem (isolated agent memory) with modern tools (MCP, vector search, embeddings). If it solves the problem of 4 agents not sharing memory, it has already won.

A. Appendix A: Complete config.yaml

The config.yaml file in the project root directory controls server, storage, cache, embedding, and backend configuration.

```
agora:
  name: "Agora"
  version: "0.4.0"

storage:
  chroma_path: "~/.agora/chroma"
  db_path: "~/.agora/agora.db"

cache:
  l1_max_entries: 1000
  l1_ttl_seconds: 300
  l2_max_entries: 10000
  l2_ttl_seconds: 86400

embedding:
  provider: "sentence-transformers"
  model: "all-MiniLM-L6-v2"

backends:
  - name: "github"
    transport: "stdio"
    command: ["npx", "-y", "@modelcontextprotocol/server-github"]
    description: "GitHub API: issues, PRs, repos, code search"
    read_only: false
    timeout_seconds: 15
    max_retries: 3
    retry_delay: 1.0
    rate_limit_rps: 5
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
  - name: "playwright"
    transport: "stdio"
    command: ["npx", "@playwright/mcp@latest"]
    description: "Browser automation: navigate, click, screenshot, forms"
    read_only: true
    timeout_seconds: 30
    max_retries: 2
    retry_delay: 0.5
    rate_limit_rps: 2
```

B. Appendix B: Project Structure

```
mcp-agora/
|-- pyproject.toml              # Dipendenze, build, entry point "agora"
|-- config.yaml                 # Configurazione server + backend
|-- agora/
|   |-- main.py                 # Entry point
|   |-- server.py               # FastMCP + 8 tools
|   |-- config.py               # YAML loader
|   |-- registry.py             # BackendRegistry
|   |-- connectors/
|   |   |-- base.py             # BackendConnector ABC
|   |   |-- stdio.py            # StdioConnector
|   |   +-- http.py             # HttpConnector
|   |-- routing/
|   |   +-- router.py           # Semantic + exact router
|   |-- embedding/
|   |   |-- base.py             # EmbeddingProvider ABC
|   |   +-- sentence.py         # sentence-transformers
|   |-- memory/
|   |   +-- vector_store.py     # ChromaDB wrapper
|   |-- cache/
|   |   |-- l1_memory.py        # TTLCache
|   |   +-- l2_cache.py         # SQLite cache
|   +-- db/
|       +-- database.py         # SQLite schema + ops
|-- tests/
|   |-- test_embedding.py       # 3 tests
|   |-- test_memory.py          # 5 tests
|   |-- test_cache.py           # 5 tests
|   |-- test_l2_cache.py        # 7 tests
|   |-- test_provenance.py      # 7 tests
|   |-- test_protocol.py        # 6 tests
|   |-- test_routing.py         # 8 tests
|   |-- test_connectors.py      # 9 tests
|   |-- test_mcp_smoke.py       # 8 tests
|   +-- _echo_server.py
|-- docs/
|   +-- generate_pdf.py
|-- README.md
+-- LICENSE (MIT)
```

C. Appendix C: Commit History

Commit history on the master branch of the GitHub repository.

```
14dbdc5  Initial commit: MCP Agora Phase 1 complete
e209f40  Add MIT License
005590b  Add agora usage instructions to AGENTS.md
5a64014  Phase 2: Routing + Backend Connectors
12e54ae  docs: update README and AGENTS.md for Phase 2
0c36648  feat: HTTP connector, read_only, broadcast, timeout
b03abbe  fix: ReadOnlyBlockedError public, parallel broadcast
dab75a2  feat: Phase 3 cross-agent memory
7f01873  docs: update ARCHITECTURE.md with Phase 2/3 details, add ruff linter, generate PDF
c4aa01c  feat: Phase 4 robustness - logging, health check, retry, rate limit, graceful tests
40e86f6  fix: non-blocking startup + WarmingUpError grace
90765e8  chore: add *.log and chroma.sqlite3 to .gitignore
```